UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS2023 Algoritmos y Estructuras de Datos
Listas
Semana 3

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

24 de agosto de 2026

¿Qué aprenderemos hoy?

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El problema que los arreglos **no** resuelven bien

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El problema que los arreglos **no** resuelven bien
- Lista Enlazada Simple

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El problema que los arreglos **no** resuelven bien
- Lista Enlazada Simple
- Lista Doblemente Enlazada

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El problema que los arreglos **no** resuelven bien
- Lista Enlazada Simple
- Lista Doblemente Enlazada
- Lista Circular

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El problema que los arreglos **no** resuelven bien
- Lista Enlazada Simple
- Lista Doblemente Enlazada
- Lista Circular
- Comparando con arreglos y vectores

¿Qué aprenderemos hoy?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- El problema que los arreglos **no** resuelven bien
- Lista Enlazada Simple
- Lista Doblemente Enlazada
- Lista Circular
- Comparando con arreglos y vectores
- Aplicaciones

El problema

Repaso: lo que ya sabemos hacer bien

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

De la semana pasada

Con `vector` logramos insertar/eliminar al **final** en $O(1)$ amortizado, gracias al crecimiento dinámico (duplicar la capacidad cuando se llena). El acceso por índice siempre fue $O(1)$.

Repaso: lo que ya sabemos hacer bien

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

De la semana pasada

Con `vector` logramos insertar/eliminar al **final** en $O(1)$ amortizado, gracias al crecimiento dinámico (duplicar la capacidad cuando se llena). El acceso por índice siempre fue $O(1)$.

¡Piénsalo!

¿Qué complejidad tiene insertar o eliminar un elemento en **medio** de un arreglo o `vector`? ¿Por qué?

La razón de fondo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Memoria contigua

Un arreglo guarda sus elementos en posiciones de memoria **consecutivas**. Insertar en medio implica **desplazar** todos los elementos posteriores una posición — inevitablemente $O(n)$, sin importar qué tan ingenioso sea el truco (el crecimiento amortizado ayuda al final, no al medio).

La razón de fondo

Memoria contigua

Un arreglo guarda sus elementos en posiciones de memoria **consecutivas**. Insertar en medio implica **desplazar** todos los elementos posteriores una posición — inevitablemente $O(n)$, sin importar qué tan ingenioso sea el truco (el crecimiento amortizado ayuda al final, no al medio).

La idea de hoy

Romper la contigüidad: que cada elemento sepa **dónde** está el siguiente, sin que tengan que estar uno junto al otro en memoria. Eso es exactamente una **lista enlazada**.

Lista Enlazada Simple

El TDA Lista Enlazada Simple

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Una colección de **nodos**, donde cada nodo guarda un **dato** y un **puntero** (referencia) al siguiente nodo. La lista se referencia por un puntero a su primer nodo, la **cabeza** (*head*); el último nodo apunta a "nada" (`nullptr`).

El TDA Lista Enlazada Simple

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

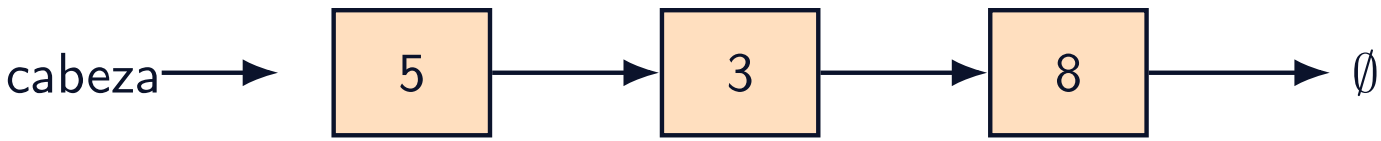
Definición

Una colección de **nodos**, donde cada nodo guarda un **dato** y un **puntero** (referencia) al siguiente nodo. La lista se referencia por un puntero a su primer nodo, la **cabeza** (*head*); el último nodo apunta a "nada" (`nullptr`).

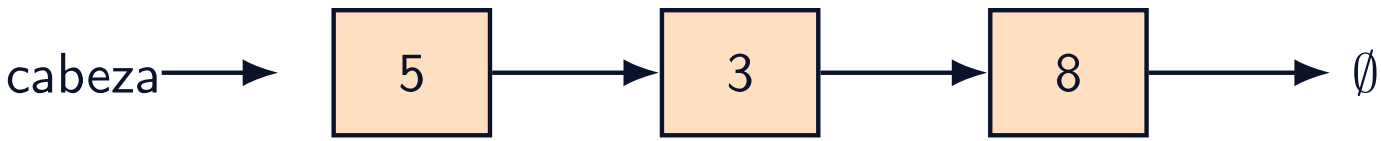
A diferencia de un arreglo

Los nodos **no** necesitan estar en posiciones consecutivas de memoria: pueden estar dispersos, y la lista los conecta con punteros.

Visualizando una lista enlazada simple



Visualizando una lista enlazada simple



Cada flecha es el puntero `siguiente` guardado **dentro** del nodo anterior — no es una posición de memoria calculable, hay que **seguir** el puntero.

Operaciones soportadas

Operación	Complejidad	Nota

Operaciones soportadas

Operación	Complejidad	Nota
Insertar al inicio	$O(1)$	solo reconectar la cabeza

Operaciones soportadas

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Operación	Complejidad		Nota
Insertar al inicio	$O(1)$		solo reconectar la cabeza
Insertar al final	$O(1)$ $O(n)$	/	$O(1)$ solo si guardamos un puntero *tail*

Operaciones soportadas

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Operación	Complejidad		Nota
Insertar al inicio	$O(1)$		solo reconectar la cabeza
Insertar al final	$O(1)$ $O(n)$	/	$O(1)$ solo si guardamos un puntero *tail*
Insertar dado un puntero	$O(1)$		si ya tienes el nodo en la mano

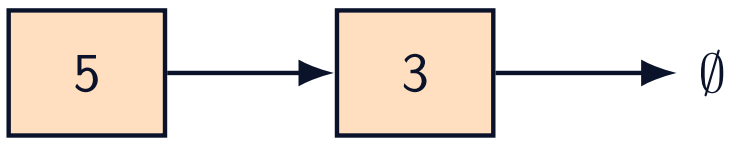
Operaciones soportadas

Operación	Complejidad		Nota
Insertar al inicio	$O(1)$		solo reconectar la cabeza
Insertar al final	$O(1)$ $O(n)$	/	$O(1)$ solo si guardamos un puntero *tail*
Insertar dado un puntero	$O(1)$		si ya tienes el nodo en la mano
Buscar / acceder a la posición k	$O(n)$		hay que **recorrer** desde la cabeza

Insertar al inicio: por qué es $O(1)$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

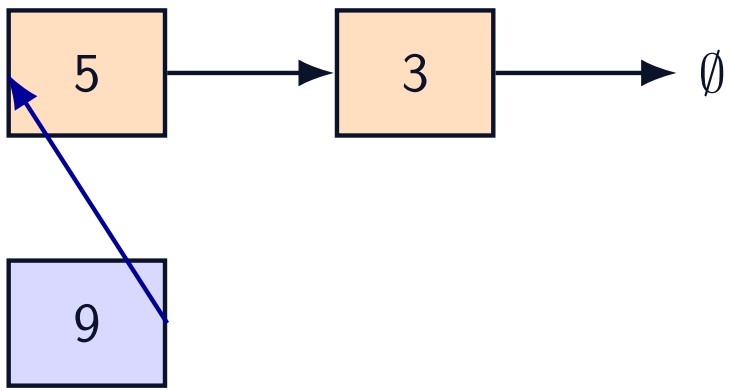
`push_front(9)`



Insertar al inicio: por qué es $O(1)$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

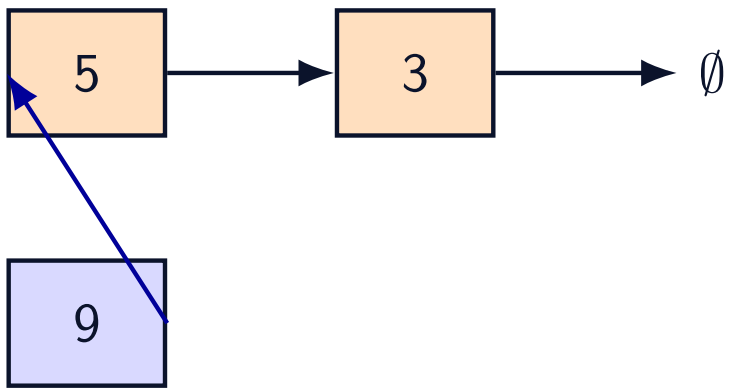
`push_front(9)`



Insertar al inicio: por qué es $O(1)$

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

`push_front(9)`



Solo se crea el nodo nuevo, se apunta al viejo primer nodo, y se mueve la cabeza — **nada más se mueve**, a diferencia de insertar al inicio de un arreglo.

La limitación: eliminar necesita el nodo *anterior*

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El problema

Para eliminar el nodo x, hay que reconectar el puntero del nodo **anterior** para que salte a x — pero en una lista simple **no hay forma de ir hacia atrás**: solo tenemos el puntero `siguiente`.

La limitación: eliminar necesita el nodo *anterior*

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El problema

Para eliminar el nodo x, hay que reconectar el puntero del nodo **anterior** para que salte a x — pero en una lista simple **no hay forma de ir hacia atrás**: solo tenemos el puntero `siguiente`.

¡Piénsalo!

Si solo tienes un puntero al nodo x (no al anterior), ¿en $O(1)$ o $O(n)$ puedes eliminarlo de una lista simple?

La limitación: eliminar necesita el nodo *anterior*

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El problema

Para eliminar el nodo x, hay que reconectar el puntero del nodo **anterior** para que salte a x — pero en una lista simple **no hay forma de ir hacia atrás**: solo tenemos el puntero `siguiente`.

¡Piénsalo!

Si solo tienes un puntero al nodo x (no al anterior), ¿en $O(1)$ o $O(n)$ puedes eliminarlo de una lista simple?

En general, $O(n)$: hay que recorrer la lista desde la cabeza **buscando** quién apunta a x, para poder reconectarlo.

Lista Doblemente Enlazada

El TDA Lista Doblemente Enlazada

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Igual que la simple, pero cada nodo guarda **dos** punteros: `siguiente` **y** `anterior`. Se puede recorrer en ambas direcciones.

El TDA Lista Doblemente Enlazada

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

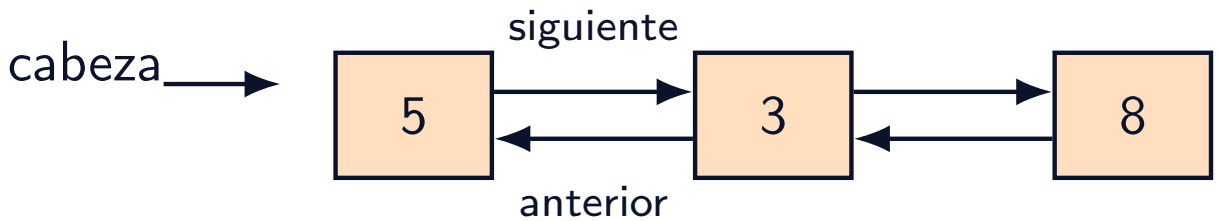
Definición

Igual que la simple, pero cada nodo guarda **dos** punteros: `siguiente` **y** `anterior`. Se puede recorrer en ambas direcciones.

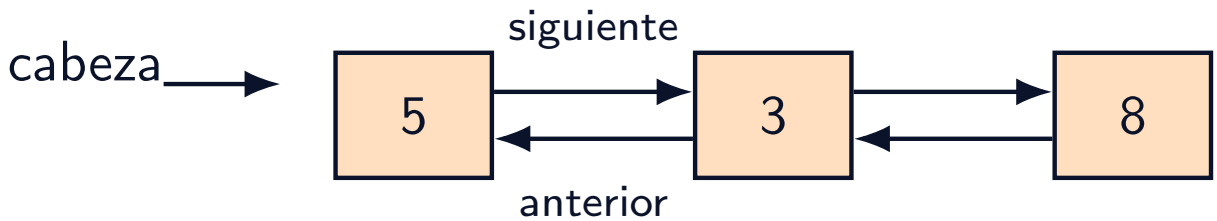
La ventaja que resuelve la limitación anterior

Si ya tienes un puntero al nodo x, también tienes **directamente** su anterior — eliminar x es $O(1)$, sin necesidad de buscarlo.

Visualizando una lista doblemente enlazada



Visualizando una lista doblemente enlazada



El costo: el doble de punteros por nodo, y el doble de trabajo al insertar/eliminar (mantener consistentes **ambos** enlaces).

Operaciones soportadas

Operación	Complejidad	Nota

Operaciones soportadas

Operación	Complejidad	Nota
Insertar al inicio/final	$O(1)$	con punteros *head* y *tail*

Operaciones soportadas

Operación	Complejidad	Nota
Insertar al inicio/final	$O(1)$	con punteros *head* y *tail*
Eliminar dado un puntero al nodo	$O(1)$	**ya no** hace falta buscar el anterior

Operaciones soportadas

Operación	Complejidad	Nota
Insertar al inicio/final	$O(1)$	con punteros *head* y *tail*
Eliminar dado un puntero al nodo	$O(1)$	**ya no** hace falta buscar el anterior
Recorrer hacia adelante o atrás	$O(n)$	en cualquiera de las dos direcciones

Operaciones soportadas

Operación	Complejidad	Nota
Insertar al inicio/final	$O(1)$	con punteros *head* y *tail*
Eliminar dado un puntero al nodo	$O(1)$	**ya no** hace falta buscar el anterior
Recorrer hacia adelante o atrás	$O(n)$	en cualquiera de las dos direcciones
Buscar / acceder a la posición k	$O(n)$	sigue sin haber acceso aleatorio

Lista Circular

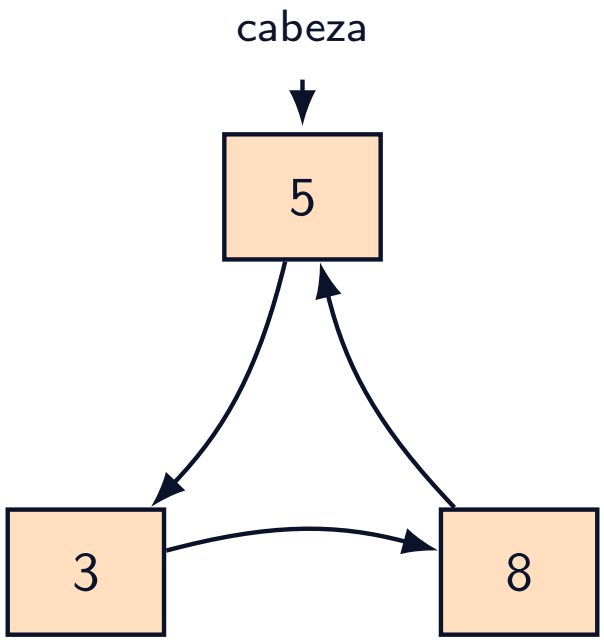
El TDA Lista Circular

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

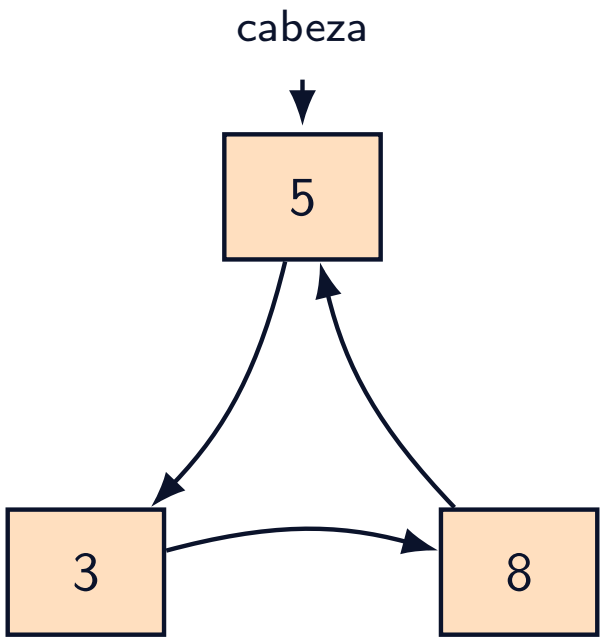
Definición

En vez de que el último nodo apunte a $\emptyset$, apunta **de vuelta** al primero: la lista "se cierra" en un círculo. Puede ser circular **simple** (un puntero por nodo) o circular **doble** (dos punteros, como la doblemente enlazada, pero cerrada).

Visualizando una lista circular simple



Visualizando una lista circular simple



No hay "último" nodo con puntero a $\emptyset$: **todos** los nodos tienen un siguiente válido.

¡Piénsalo!

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Cuándo paras de recorrer?

Si recorres una lista circular con `while(p != nullptr)`, ¿qué pasa? ¿Cómo debería ser la condición de parada correcta?

¡Piénsalo!

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Cuándo paras de recorrer?

Si recorres una lista circular con `while(p != nullptr)`, ¿qué pasa? ¿Cómo debería ser la condición de parada correcta?

La respuesta

`p` **nunca** llega a $\emptyset$ — el ciclo **nunca termina**. Hay que parar cuando `p` vuelve a ser la cabeza (o llevar un contador de cuántos nodos hay).

Comparando con arreglos y vectores

Tabla comparativa

	Arreglo/vector	Lista simple	Lista doble

Tabla comparativa

	Arreglo/vector	Lista simple	Lista doble
Acceso por índice	$O(1)$	$O(n)$	$O(n)$

Tabla comparativa

	Arreglo/vector	**Lista simple**	**Lista doble**
Acceso por índice	$O(1)$	$O(n)$	$O(n)$
Insertar/eliminar al inicio	$O(n)$ / $O(1)$ am.	$O(1)$	$O(1)$

Tabla comparativa

	Arreglo/vector	**Lista simple**	**Lista doble**
Acceso por índice	$O(1)$	$O(n)$	$O(n)$
Insertar/eliminar al inicio	$O(n)$ / $O(1)$ am.	$O(1)$	$O(1)$
Insertar/eliminar al final	$O(1)$ am.	$O(1)$ con *tail*	$O(1)$

Tabla comparativa

	Arreglo/vector	**Lista simple**	**Lista doble**
Acceso por índice	$O(1)$	$O(n)$	$O(n)$
Insertar/eliminar al inicio	$O(n)$ / $O(1)$ am.	$O(1)$	$O(1)$
Insertar/eliminar al final	$O(1)$ am.	$O(1)$ con *tail*	$O(1)$
Insertar/eliminar en medio (con puntero)	$O(n)$	$O(1)$	$O(1)$

Tabla comparativa

	Arreglo/vector	**Lista simple**	**Lista doble**
Acceso por índice	$O(1)$	$O(n)$	$O(n)$
Insertar/eliminar al inicio	$O(n)$ / $O(1)$ am.	$O(1)$	$O(1)$
Insertar/eliminar al final	$O(1)$ am.	$O(1)$ con *tail*	$O(1)$
Insertar/eliminar en medio (con puntero)	$O(n)$	$O(1)$	$O(1)$
Eliminar dado el puntero al nodo	— (no aplica)	$O(n)$	$O(1)$

Tabla comparativa

	Arreglo/vector	**Lista simple**	**Lista doble**
Acceso por índice	$O(1)$	$O(n)$	$O(n)$
Insertar/eliminar al inicio	$O(n)$ / $O(1)$ am.	$O(1)$	$O(1)$
Insertar/eliminar al final	$O(1)$ am.	$O(1)$ con *tail*	$O(1)$
Insertar/eliminar en medio (con puntero)	$O(n)$	$O(1)$	$O(1)$
Eliminar dado el puntero al nodo	— (no aplica)	$O(n)$	$O(1)$
Memoria extra por elemento	ninguna	1 puntero	2 punteros

¡Piénsalo!

Si insertar/eliminar en medio es $O(1)$...

...dado un puntero, ¿por qué no reemplazamos **todos** los arreglos por listas enlazadas?

¡Piénsalo!

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Si insertar/eliminar en medio es $O(1)$...

...dado un puntero, ¿por qué no reemplazamos **todos** los arreglos por listas enlazadas?

Dos razones

(1) No hay acceso aleatorio: **llegar** hasta esa posición para tener el puntero en la mano ya cuesta $O(n)$, salvo que ya lo tengas guardado de antes. (2) Los nodos están **dispersos** en memoria: mucho peor uso de la caché que un arreglo contiguo, en la práctica.

La regla práctica

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Cuándo usar cada uno?

Arreglo/vector: cuando necesitas acceso por índice, o recorres secuencialmente aprovechando la localidad de memoria (el caso más común).
Lista enlazada: cuando insertas o eliminas mucho en posiciones **ya conocidas** (tienes el puntero en mano) y no necesitas acceso aleatorio.

Aplicaciones

Casos de uso reales

Casos de uso reales

- **Historial de navegación** (atrás/adelante): lista doble, moviéndose entre nodos sin reconstruir nada.

Casos de uso reales

- **Historial de navegación** (atrás/adelante): lista doble, moviéndose entre nodos sin reconstruir nada.
- **Deshacer/rehacer** en un editor: similar, con un puntero a la posición "actual" que se mueve.

Casos de uso reales

- **Historial de navegación** (atrás/adelante): lista doble, moviéndose entre nodos sin reconstruir nada.
- **Deshacer/rehacer** en un editor: similar, con un puntero a la posición "actual" que se mueve.
- **Reproductor de música en modo repetir**: lista circular, para que después de la última canción se vuelva a la primera sin caso especial.

Casos de uso reales

- **Historial de navegación** (atrás/adelante): lista doble, moviéndose entre nodos sin reconstruir nada.
- **Deshacer/rehacer** en un editor: similar, con un puntero a la posición "actual" que se mueve.
- **Reproductor de música en modo repetir**: lista circular, para que después de la última canción se vuelva a la primera sin caso especial.
- **Planificación *round-robin***: dar turnos de CPU en un círculo de procesos, avanzando un nodo a la vez.

Casos de uso reales

- **Historial de navegación** (atrás/adelante): lista doble, moviéndose entre nodos sin reconstruir nada.
- **Deshacer/rehacer** en un editor: similar, con un puntero a la posición "actual" que se mueve.
- **Reproductor de música en modo repetir**: lista circular, para que después de la última canción se vuelva a la primera sin caso especial.
- **Planificación *round-robin***: dar turnos de CPU en un círculo de procesos, avanzando un nodo a la vez.
- Las implementaciones reales de `std::list` (C++, doble) y `std::forward_list` (C++, simple) — las construiremos nosotros mismos la próxima clase.

Resumen de la sesión



Resumen de la sesión

- Una **lista enlazada** rompe la contigüidad de memoria: cada nodo sabe dónde está el siguiente (y, en la doble, también el anterior).

Resumen de la sesión

- Una **lista enlazada** rompe la contigüidad de memoria: cada nodo sabe dónde está el siguiente (y, en la doble, también el anterior).
- Ganamos $O(1)$ para insertar/eliminar en cualquier posición **ya conocida** — a cambio de perder el acceso aleatorio ($O(n)$ para llegar a una posición).

Resumen de la sesión

- Una **lista enlazada** rompe la contigüidad de memoria: cada nodo sabe dónde está el siguiente (y, en la doble, también el anterior).
- Ganamos $O(1)$ para insertar/eliminar en cualquier posición **ya conocida** — a cambio de perder el acceso aleatorio ($O(n)$ para llegar a una posición).
- La versión **doble** permite eliminar en $O(1)$ dado solo el puntero al nodo, y recorrer en ambas direcciones.

Resumen de la sesión

- Una **lista enlazada** rompe la contigüidad de memoria: cada nodo sabe dónde está el siguiente (y, en la doble, también el anterior).
- Ganamos $O(1)$ para insertar/eliminar en cualquier posición **ya conocida** — a cambio de perder el acceso aleatorio ($O(n)$ para llegar a una posición).
- La versión **doble** permite eliminar en $O(1)$ dado solo el puntero al nodo, y recorrer en ambas direcciones.
- La versión **circular** cierra el último nodo con el primero — útil cuando no hay un "final" natural.

Resumen de la sesión

- Una **lista enlazada** rompe la contigüidad de memoria: cada nodo sabe dónde está el siguiente (y, en la doble, también el anterior).
- Ganamos $O(1)$ para insertar/eliminar en cualquier posición **ya conocida** — a cambio de perder el acceso aleatorio ($O(n)$ para llegar a una posición).
- La versión **doble** permite eliminar en $O(1)$ dado solo el puntero al nodo, y recorrer en ambas direcciones.
- La versión **circular** cierra el último nodo con el primero — útil cuando no hay un "final" natural.
- **Próxima clase:** implementamos las tres con punteros reales en C++ (y vemos que también se pueden simular sin punteros, usando un arreglo).

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias